

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ
И ИНФОРМАТИКИ»

Кафедра
вычислительных
систем

КУРСОВАЯ РАБОТА
по дисциплине «Архитектура ЭВМ»

Выполнили:
студенты группы
ИПЗ11:
Подкорытова А.В.
Пудовкин И.С.
Фатеева К.А.

Проверил(а):

Новосибирск, 2025

Оглавление

Постановка задачи.....	3
Блок-схемы алгоритмов.....	5
Программная реализация.....	7
Результаты тестирования.....	11
Заключение.....	12
Список используемой литературы.....	13

Постановка задачи

- Разработать транслятор с языка Simple Assembler. Итог работы транслятора – бинарный файл с образом оперативной памяти Simple Computer, который можно загрузить в модель и выполнить;
- Доработать модель Simple Computer – реализовать алгоритм работы блока «L2-кэш команд и данных» и модифицировать работу контроллера оперативной памяти и обработчика прерываний таким образом, чтобы учитывался простой процессора при прямом доступе к оперативной памяти;

Транслятор с языка Simple Assembler

Разработка программ для Simple Computer может осуществляться с использованием низкоуровневого языка Simple Assembler. Для того чтобы программа могла быть обработана Simple Computer необходимо реализовать транслятор, переводящий текст Simple Assembler в бинарный формат, которым может быть считан консолью управления. Пример программы на Simple Assembler:

READ 09	;(Ввод A)
00 READ 10	;(Ввод B)
01 LOAD 09	;(Загрузка A в аккумулятор)
02 SUB 10	;(Отнять B)
03 JNEG 07	;(Переход на 07, если отрицательное)
04 WRITE 09	;(Вывод A)
05 HALT 00	;(Останов)
06 WRITE 10	;(Вывод B)
HALT 00	;(Останов)
09 = +0000	;(Переменная A)
10 = +9999	;(Переменная B)

Программа транслируется по строкам, задающим значение одной ячейки памяти. Каждая строка состоит как минимум из трех полей: адрес ячейки памяти, команда (символьное обозначение), операнд. Четвертым полем может быть указан комментарий, который обязательно должен начинаться с символа точка с запятой. Название команд представлено в таблице 1. Дополнительно используется команда =, которая явно задает значение ячейки памяти в формате вывода его на экран консоли (+XXXX).

Команда запуска транслятора должна иметь вид: sat файл.sa файл.o, где файл.sa – имя файла, в котором содержится программа на Simple Assembler, файл.o – результат трансляции.

Оформление отчёта по курсовой работе

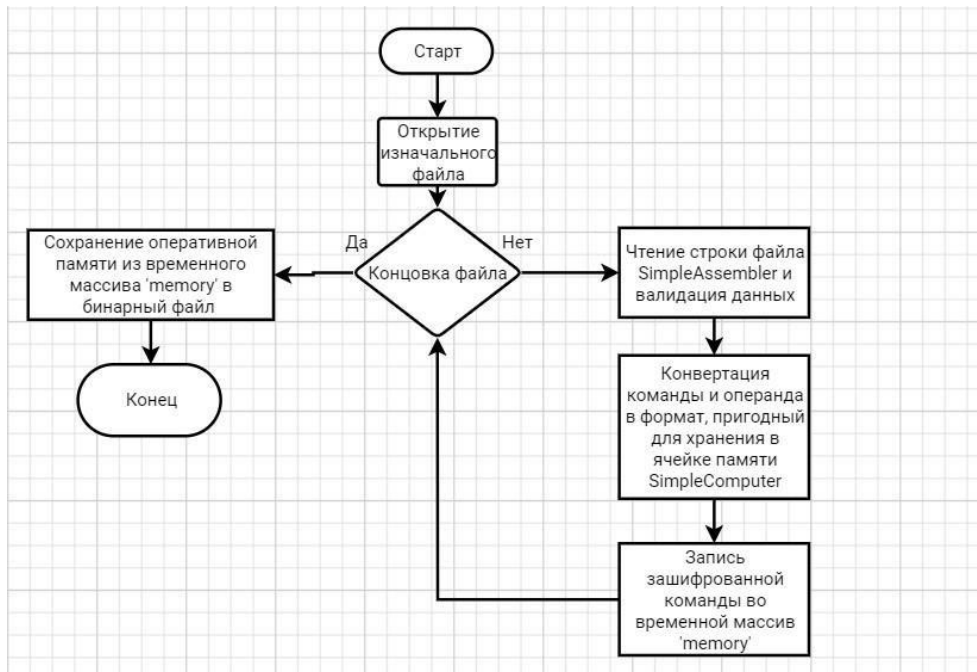
Отчет о курсовой работе представляется в виде пояснительной записки (ПЗ), к которой прилагается диск с разработанным программным обеспечением. В пояснительную записку должны входить:

- титульный лист;
- полный текст задания к курсовой работе;
- реферат (объем ПЗ, количество таблиц, рисунков, схем, программ, приложений, краткая характеристика и результаты работы);
- содержание:
 - постановка задачи исследования;
 - блок-схемы используемых алгоритмов;
 - программная реализация;
 - результаты проведенного исследования;
 - выводы;
- список использованной литературы;
- подпись, дата.

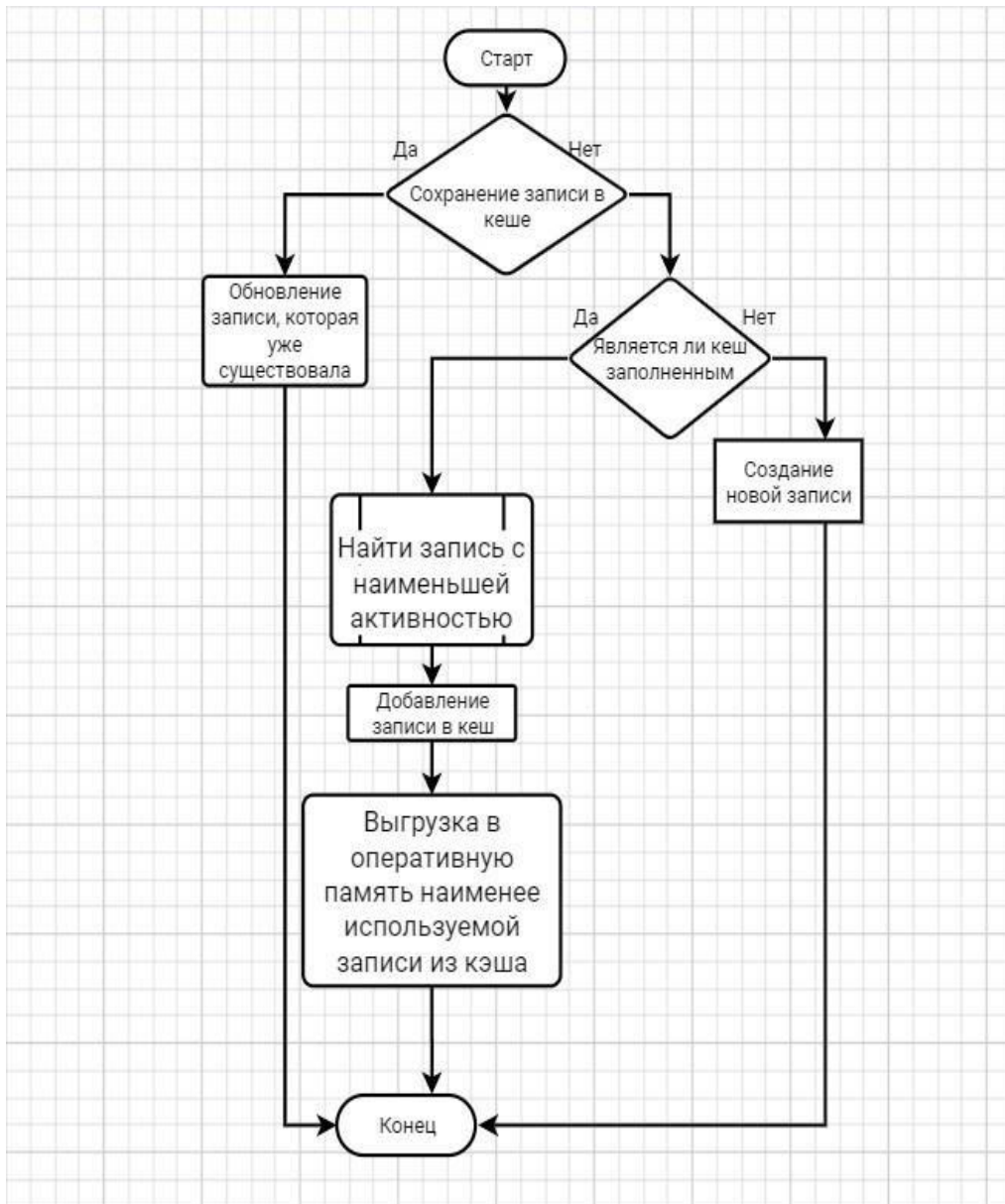
Пояснительная записка должна быть оформлена на листах формата А4, имеющих поля. Все листы следует сброшюровать и пронумеровать.

Блок-схемы алгоритмов

SimpleAssembler:



Кеш процессора:



Программная реализация

SimpleAssembler:

sat.c:

Программа `sat.c` реализует простой ассемблер, который преобразует команды из текстового файла с расширением `.sa` в бинарный формат и записывает их в выходной файл с расширением `.o`. Программа начинается с подключения стандартных библиотек: `stdio.h` для ввода-вывода, `stdlib.h` для функций общего назначения, `string.h` для работы со строками и `ctype.h` для работы с символами. Определяется константа `MEM_SIZE` равная 128, задающая размер памяти для хранения команд.

Функция **`opcode_of`** принимает строку `mn`, представляющую мнемонику команды, и возвращает соответствующий код операции в шестнадцатеричном формате. Используется `strcmp` для сравнения входной строки с известными мнемониками: `NOP (0x00)`, `CPUINFO (0x01)`, `READ (0x0A)`, `WRITE (0x0B)`, `LOAD (0x14)`, `STORE (0x15)`, `ADD (0x1E)`, `SUB (0x1F)`, `DIVIDE (0x20)`, `MUL (0x21)`, `JUMP (0x28)`, `JNEG (0x29)`, `JZ (0x2A)`, `HALT (0x2B)`, `NOT (0x33)`, `LOGRC (0x44)`, `RCCL (0x45)`, `MOVA (0x47)`, `MOVR (0x48)`, `SUBC (0x4C)`. Если мнемоника не найдена, функция возвращает -1, что указывает на ошибку.

Функция **`main`** принимает аргументы командной строки `argc` и `argv`. Проверяется, что `argc` равно 3, то есть переданы имя программы, входной и выходной файлы. Если это не так, выводится сообщение об использовании ("`Usage: sat input.sa output.o`") и программа завершается с кодом `EXIT_FAILURE`. Переменные `infile` и `outfile` получают значения `argv[1]` и `argv[2]` соответственно. Входной файл открывается для чтения с помощью `fopen`. Если открытие не удалось, выводится сообщение об ошибке через `fprintf` и программа завершается.

Инициализируется массив `memory` размером `MEM_SIZE`, заполненный нулями, для хранения команд и данных. В цикле `while` с помощью `fgets` читаются строки из входного файла в буфер `line`. Указатель `p` пропускает начальные пробельные символы в строке. Если строка пустая, начинается с символа комментария `';` или `'\n'`, она пропускается. Из строки извлекается адрес `addr` как шестнадцатеричное число с помощью `strtoul`. Если адрес находится вне диапазона `[0, MEM_SIZE)`, выводится сообщение об ошибке и строка пропускается.

Далее анализируется содержимое строки. Если после адреса идет символ '=', извлекается значение val как десятичное число с помощью strtol. Если val отрицательное, оно преобразуется в 15-битное число с установкой 14-го бита (знаковый бит) и сохранением модуля в младших 14 битах, иначе значение обрезаается до 15 бит и записывается в memory[addr]. Если вместо '=' идет команда, с помощью sscanf извлекаются мнемоника и операнд в шестнадцатеричном формате. Если синтаксис неверный, выводится сообщение об ошибке.

Для мнемоники вызывается opcode_of, чтобы получить код операции. Если возвращается -1, выводится сообщение об неизвестной мнемонике. Проверяется, что операнд находится в диапазоне [0, 127]. Если это не так, выводится сообщение об ошибке. Код операции сдвигается на 7 бит влево, объединяется с операндом побитовым ИЛИ, результат обрезаается до 15 бит и сохраняется в memory[addr].

После обработки всех строк входной файл закрывается. Выходной файл открывается для записи в бинарном режиме с помощью fopen. Если открытие не удалось, выводится сообщение об ошибке и программа завершается. Массив memory записывается в выходной файл с помощью fwrite. Если количество записанных элементов не равно MEM_SIZE, выводится сообщение об ошибке записи. Выходной файл закрывается, и программа завершает выполнение с кодом EXIT_SUCCESS, если все операции выполнены успешно.

Кеш процессора:

Cache.c:

Файл Cache.c реализует систему кэша процессора с использованием структуры **CacheLine**, которая включает поля: tag (адрес начала строки, -1 если свободно), data (массив данных строки размером LINE_SIZE), valid (флаг валидности), dirty (флаг измененности) и last_used (время последнего использования для алгоритма LRU). Определяются константы CACHE_LINES (5) и LINE_SIZE (10), а также глобальная переменная current_time для отслеживания времени доступа. Массив cache и счетчик cache_counter инициализируются для работы с кэшем.

Функция **initCache** обнуляет `current_time` и инициализирует каждую строку кэша: `tag` устанавливается в -1, `valid` и `dirty` сбрасываются в 0, `last_used` обнуляется, а массив `data` заполняется нулями.

Функция **findCacheLine** ищет строку кэша по адресу, вычисляя начальный адрес линии (`line_start`) как `address` минус остаток от деления на `LINE_SIZE`. Проходится по всем строкам кэша, проверяя совпадение `tag` и `valid`. Если строка найдена, обновляется `last_used` и возвращается её индекс, иначе -1.

Функция **findFreeCacheLine** ищет первую свободную строку (где `valid` равно 0) и возвращает её индекс, или -1, если свободных строк нет.

Функция **findLRUCacheLine** определяет индекс строки с наименьшим `last_used`, используя алгоритм LRU (Least Recently Used), и возвращает его для замещения.

Функция **loadCacheLine** загружает данные по адресу в кэш. Проверяется допустимость адреса (0-127), иначе устанавливается флаг ошибки `FLAG_M`. Вычисляется `line_start`, ищется строка в кэше с помощью `findCacheLine`. Если строка найдена, возвращается её индекс. Если нет свободных строк, используется `findLRUCacheLine`. Если замещаемая строка `dirty` и `valid`, её данные сохраняются в RAM. Новая строка инициализируется: `tag` устанавливается в `line_start`, `valid` в 1, `dirty` в 0, `last_used` обновляется, а `data` загружается из RAM по соответствующим адресам.

Функция **printCache** отображает содержимое кэша. Рисуются рамка с заголовком "Кэш процессора". Ищется строка по адресу с помощью `findCacheLine`. Если промах (miss), строка загружается через `loadCacheLine`, устанавливается задержка `Takt = 10`. При попадании (hit) `Takt = 1`. Задержка отображается с помощью цикла `sleep`. Для каждой строки кэша выводится `tag`, данные в формате с знаком, кодом и операндом, а если `dirty`, добавляется '*'. Пустые строки отображаются как "-1" с пробелами.

Функция **cacheRead** читает значение по адресу. Проверяется допустимость адреса, иначе устанавливается `FLAG_M`. Ищет строку в кэше, при промахе загружает её через `loadCacheLine`. Возвращает значение из `data` по смещению `address % LINE_SIZE`.

Функция **cacheWrite** записывает значение по адресу. Проверяется допустимость адреса, иначе устанавливается `FLAG_M`. Ищет строку в кэше,

при промахе загружает её. Записывает значение в data, устанавливает dirty в 1, обновляет last_used, и синхронизирует с RAM.

Функция **clearCache** вызывает initCache и сбрасывает cache_counter.

Функция **clearCacheDisplay** очищает отображение кэша, рисуя рамку и заполняя строки пробелами.

Input_Output.c:

Функция **handle_input** управляет интерфейсом: обрабатывает клавиши (перемещение, редактирование, загрузка/сохранение, сброс, запуск), обновляет отображение и синхронизирует с кэшем и RAM. При сбросе (ключ I) вызывает clearCache. В автоматическом режиме (ключ R) может загружать строку кэша через loadCacheLine при промахе.

Результат трансляции с SimpleAssembler:

[illegible]

Заключение

В ходе выполнения курсовой работы были достигнуты следующие результаты:

1. Реализовано усовершенствование модели Simple Computer с интеграцией блока “Кеш процессора”.
2. Создан транслятор с языка Simple Assembler, позволяющий легко осуществлять перевод ассемблерного кода в машинные команды, обеспечивая высокую производительность программы.

Программы успешно протестированы, а результаты тестов подробно описаны в отчёте.

Список использованной литературы

1. ЭВМ и ПЕРИФЕРИЙНЫЕ УСТРОЙСТВА [Учебное пособие]. С.Н. Мамоиленко О.В. Молдованова.
2. Изучаем программирование на С [книга]. Дон Гриффитс, Дэвид Гриффитс.
3. Язык программирования С [книга]. Брайан Керниган и Денис Ритчи.
4. Программирование на С в примерах и задачах [книга]. Васильев А. Н.